

Temenos OFS String Injection: Revealing a Hidden Financial Attack Vector

Temenos OFS String Injection: Revealing a Hidden Financial Attack Vector - Omar Elshopky (3l5h0pky), Medium.

- Published: 2025-12-09
- Original: <<https://medium.com/@omarelshopky/temenos-ofs-field-injection-revealing-a-hidden-financial-attack-vector-44df0e2bef9d>>
- Preserved from: <https://medium.com/@omarelshopky/temenos-ofs-field-injection-revealing-a-hidden-financial-attack-vector-44df0e2bef9d> (live) on 2026-08-09
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Penetration Testing

Fintech

Cybersecurity

Temenos OFS Field Injection: Revealing a Hidden Financial Attack Vector

TL;DR

[[[image: Omar Elshopky \(3l5h0pky\)](#)]](https://medium.com/@omarelshopky?source=post_page---byline--44df0e2bef9d-----)

[Omar Elshopky \(3l5h0pky\)](#)

During a pentest of an API integrated with Temenos using OFS, I uncovered a previously undocumented attack vector that I call **OFS Field Injection**. Improperly sanitized user input was inserted directly into OFS request strings, enabling the creation of poisoned transactions and theft of funds with minimal trace. The post explains how OFS works, how this attack vector emerges, and why proper input validation is critical for secure Temenos integrations.

During an instant payment API pentesting engagement at [Spark Professional Services](#), I encountered an error response seemingly originating from the Temenos (T24) core banking system. With further research and repeated attempts, this behavior led to achieving two critical

outcomes: Unauthorized Transaction Execution with funds deducted from a victim's account and successful outward transfers without any balance deduction at all.

Since no public reference to this attack path could be found, I named it “**Temenos OFS Field Injection**” and decided to document the background, reasoning, and exploitation techniques behind this vector.

The tested APIs handled instant payment transfers, allowing users to move funds from one account to another. To demonstrate the attack without exposing client data, I created a minimal mock server containing a single endpoint `/transfer`, accepting sender account number, receiver account number, amount, and currency, returning a transfer status and transaction ID as shown below:

Internally, the endpoint relied on the core banking system to validate balance and execute transfers. Before diving into the attack, we need a brief understanding of Temenos and, specifically, the OFS component that sits at the center of this behavior.

What is Temenos T24?

Temenos is a global banking software provider delivering core banking, payment, wealth management, and other financial systems to more than 950 financial institutions worldwide. Their solutions are widely used to manage accounts, process transactions, and enable digital banking services.

At a high level, Temenos is typically divided into two main parts:

- **Temenos Transact (T24):** The core banking engine
- **Temenos Digital:** Customer-facing interfaces such as online banking portals and mobile applications

What is Open Financial Service (OFS)?

OFS (Open Financial Services) is an interface in Temenos T24 that processes both transaction commands and data queries. It is the standard way external systems integrate with T24 to fetch account information or initiate financial operations, as in this instant payment implementation.

OFS supports two formats:

- **Native format:** A comma-structured text string
- **XML format:** Mostly used in Temenos web applications

This research focuses exclusively on the **Native OFS format**, which supports two request types:

- **Transaction Requests**
- **Enquiry Requests**

OFS Native Format Syntax

A typical OFS command minimally follows this structure:

```
<OPERATION>,<OPTIONS>,<USER_INFO>,<TRANSACTION_ID>,<REQUEST_DATA>
```

- **OPERATION:** The action to perform, such as

ACCOUNT , FUNDS.TRANSFER , or ENQUIRY.SELECT .

- **OPTIONS:** Five-part string

<VERSION>/<FUNCTION>/<PROCESS_TYPE>/<GTS_CONTROL>/<NO_OF_AUTHORISERS> , where, for example:

FUNCTION = I insert/modify FUNCTION = S read FUNCTION = D delete PROCESS_TYPE = PROCESS

to commit PROCESS_TYPE = VALIDATE to verify only

- **USER_INFO:** USERNAME/PASSWORD/COMPANY_CODE where the company code is optional
- **TRANSACTION_ID:** Record ID (auto-generated if left empty)
- **REQUEST_DATA:** Multiple comma-separated field definitions in the format:

<FIELD_NAME>:<MULTI_VALUE>:<SUB_VALUE>=<FIELD_CONTENT>

A response follows:

```
<TRANSACTION_ID>//<STATUS_CODE>/NO,<RESPONSE_DATA|ERROR>
```

Where STATUS_CODE may be:

- 1 success
- -1 Error encountered during processing
- -2 Override encountered during processing
- -3 Core banking server is in offline

Enquiries always use:

```
ENQUIRY.SELECT,,<USER_INFO>,<ENQUIRY_NAME>,<REQUEST_DATA>
```

Creating a new account may look like:

```
ACCOUNT,MTD.NEW.AC/I/PROCESS,3l5h0pky/123456/123,92392,CUSTOMER:=111211,CATEGORY:=1001,CURRENCY:=E
```

Where:

- ACCOUNT the operation
- MTD.NEW.AC the version
- I for input function
- PROCESS for committing the record
- 3l5h0pky the username
- 123456 the password
- 123 the company code
- 92392 the transaction ID
- CUSTOMER:=111211,CATEGORY:=1001,CURRENCY:=EGP,ACCOUNT.TITLE.1:=TEST ACC The new account details

A successful response may contain the created account details.

```
92392//1,CUSTOMER:1:1=111211,CATEGORY:1:1=1001,ACCOUNT.TITLE:1:1=TEST ACC,SHORT.TITLE:1:1=TEST ACC,MNEMON
```

A failed request instead returns:

```
92392//1/NO,<ERROR>
```

For retrieving or deleting functions, there is no need for `<REQUEST_DATA>`

```
ACCOUNT,MTD.NEW.AC/S/PROCESS,3I5h0pky/123456/123,92392  
ACCOUNT,MTD.NEW.AC/D/PROCESS,3I5h0pky/123456/123,92392
```

<https://ofs.etelej.com> can be used to play with the T24 OFS Message for better understanding

With this basic understanding of Temenos and OFS, testing could properly begin.

Finding the Marks

While fuzzing the `/transfer` endpoint, I injected a comma `,` into all input fields. The intention was simply to observe how each field handled special characters, and the resulting server response revealed a Temenos error:

Injecting "100," caused an error that revealed the underlying core banking system.

A quick search for the error code `TAFJERR-1060` confirmed that:

- The backend system was indeed Temenos
- The injection point was interacting with OFS
- The comma had broken OFS structure

Now aware that I could influence the OFS request, I continued testing additional characters while researching the integration format. After a few hours of analysis, the behavior aligned perfectly with OFS syntax: the injected comma attempted to start a new field but failed to form the complete `<FIELD>::=<VALUE>` format, producing a parsing error.

Adding a dummy field such as `NAME:test` resulted in a new error indicating that the supplied field was not valid for the `FUNDS.TRANSFER` operation, confirming that this was the operation backing the `/transfer` endpoint:

Injecting "100,NAME:test" exposed the operation name in the response.

Important Constraint: **OFS transactions only allow **one operation per request, meaning we are locked into the existing operation structure until the injection point. With that in mind, a typical request payload around the injection point looks like:

```
FUNDS.TRANSFER,<VERSION>/I/PROCESS,<USER>/<PASS>,,<FIELDS>,DEBIT.AMOUNT::=**<INJECTION_POINT>**,<OTH
```

Next, I enumerated supported fields for `FUNDS.TRANSFER` and identified several that could directly affect transaction outcome, including:

- `TRANSACTION.TYPE` : Type of funds transfer transaction (Values: `AC` — Account Transfer, `TT` — Telegraphic Transfer, `DD` — Demand Draft, `RT` — Real Time Transfer, `CH` — Cheque)
- `DEBIT.ACCT.NO` : Account to be debited
- `DEBIT.CURRENCY` : Currency of the debit account
- `DEBIT.AMOUNT` : Amount to be debited
- `CREDIT.ACCT.NO` : Account to be credited (for account transfers)
- `CREDIT.CURRENCY` : Currency of the credit transaction
- `PROCESSING.DATE` : Date when transaction should be processed (Format YYYYMMDD)

Testing demonstrated that some fields (such as `DEBIT.AMOUNT`) enforced positional constraints, but others did not, making exploitation possible.

Scenario 1# Deduct Money from a Victim Account

While experimenting with OFS field injection, I focused on fields that directly influence the internal behavior of a `FUNDS.TRANSFER` transaction. One of the most impactful was `DEBIT.ACCT.NO` , which defines the account to be debited during the operation. Normally, this field should be set by the application to the authenticated user's account, ensuring the initiated transfer deducts funds from the correct source.

However, because user-controlled data was injected directly into the OFS request without sanitization, it was possible to override this field entirely. By injecting additional fields immediately after the position of `DEBIT.AMOUNT` , the malicious input became part of the final OFS command processed by Temenos. The only requirement was ensuring the injected segment remained syntactically valid by correctly following OFS formatting rules.

For example, injecting:

```
100,DEBIT.CURRENCY::=EGP,DEBIT.ACCT.NO::=0000000000000789
```

does three things:

- **Sets a valid currency value** (`DEBIT.CURRENCY::=EGP`) to satisfy field ordering and validation rules.
- **Introduces a new value for ** `DEBIT.ACCT.NO` ***, completely replacing the legitimate account number assigned by the application.
- **Causes the core banking system to debit the victim's account**, while the receiving account (belonging to the attacker) still receives the funds normally.

Temenos processes the transaction as if the victim legitimately initiated it themselves. The transfer completes successfully, and the system reflects a legitimate outgoing transaction from the victim's

account. No validation fails, no anomalies are raised, and from the core banking system's perspective, the transaction appears fully authorized.

The victim account 0000000000000789 will be debited instead of the sender.

This demonstrates that once the attacker can control the OFS field structure, the integrity of financial operations depends entirely on the application's ability to properly sanitize and escape input before constructing OFS messages.

The transaction executed successfully, the logs recorded normal activity, and the only forensic evidence resided in the victim's transaction list, a damaging but detectable outcome. The next scenario, however, removes even that trace.

Scenario 2# Transfers with No Balance Deduction

One field immediately raised suspicion:

```
PROCESSING.DATE
```

At first glance, you might expect setting a future processing date to either:

- Delay execution, or
- Hold the required funds until the set date

However, testing showed otherwise.

The transaction completed successfully, crediting the receiving account immediately, **while no balance was deducted or held from the sender account at all.**

This enabled unlimited transfers far beyond available balance. To validate, I:

- Set the processing date to a future value
- Waited for the actual date
- Confirmed no deduction ever occurred

Further consultation confirmed that no holds or pending entries existed in the system. The transactions appeared legitimate but left almost no trace, enabling silent fund theft.

Pushing the value further eventually reached:

```
20600831
```

Beyond which the system returned validation errors due to missing holiday calendar configuration:

```
100,DEBIT.CURRENCY::=EGP,PROCESSING.DATE::=20600831
```

The Solution

The fix is straightforward:

Sanitize user-controlled input before inserting it into OFS requests.

Prevent characters such as:

```
',' /, //, _ ;, =
```

from being interpreted as OFS syntax components unless intentionally introduced.

This case shows how missing basic input handling in critical financial systems can lead to high-impact business compromise and reminds us that even a single error message can reveal valuable insight if properly investigated.

Mnemonics: A Remaining Risk

Even after sanitization, another Temenos feature — **Mnemonics** — may introduce secondary risk. Mnemonics allow assigning aliases to accounts, such as using `OMAR` instead of `0000000000000123`. Without strict validation, this can be abused in some contexts and should be accepted only where business requirements justify it. Backend controls should ensure Mnemonics cannot bypass account-format expectations in sensitive operations.

Using a mnemonic instead of an account number to execute the transfer.

Break it right, secure it tight ;)

Archived from <https://medium.com/@omarelshopky/temenos-ofs-field-injection-revealing-a-hidden-financial-attack-vector-44df0e2bef9d>. Rendered offline from the local Markdown copy.